

The Founder's Tech Handbook

Every programming concept in Bari's stack — in plain language, with analogies, diagrams, and a “say it like this” line for when you're in the room.

How to use this. You don't need to memorise it. Skim the **five mental models** first — they're the foundation everything else hangs on. Then keep this open during the meeting and **Ctrl-F** any word the engineer uses. Each entry tells you what it *is*, a real-world **analogy**, how it's used **in Bari**, and — where it matters — exactly how to **say it** so you sound fluent without overclaiming.

CONTENTS

- | | |
|------------------------------------|--------------------------------------|
| 1. Five mental models (start here) | 6. AI & language models |
| 2. Data & how it's stored | 7. Engineering glue & practices |
| 3. The backend — making the data | 8. Bari's own vocabulary |
| 4. Testing & trust | 9. One-page cheat sheet |
| 5. The frontend — the website | 10. Sound fluent · avoid these slips |

1 Five mental models

Get these five and you can reason about almost any technical question, even ones you've never heard.

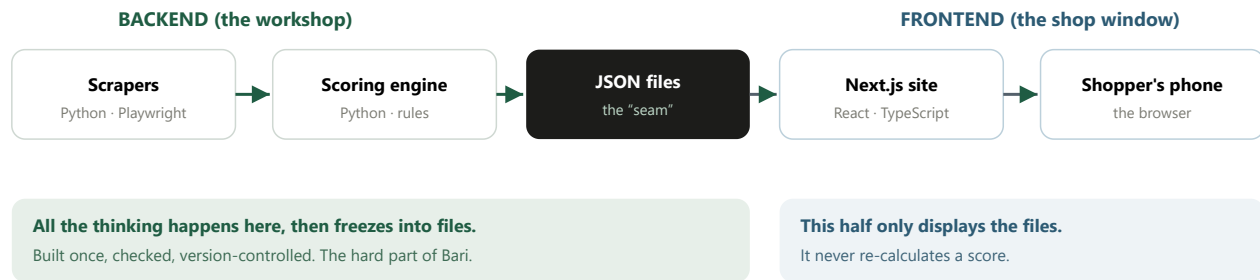


Figure A — The single most important picture: two halves joined only by JSON files. Memorise this and most questions answer themselves.

1 • Backend vs. Frontend

The **backend** is everything the user never sees — the data collection, the scoring, the logic. The **frontend** is what they *do* see — the website on their phone.

ANALOGY *A restaurant: the kitchen (backend) cooks; the dining room (frontend) serves. Diners only see the plate.*

IN BARI Backend = the Python pipeline that scores 500 products. Frontend = the Next.js website. They never mix.

2 • Deterministic vs. Probabilistic

Deterministic = same input always gives the exact same output. **Probabilistic** = the output can vary, like an AI that answers slightly differently each time.

ANALOGY *A calculator is deterministic (2+2 is always 4). Asking a person “rate this 1–10” is probabilistic (you’ll get 7, then 6).*

IN BARI The scoring engine is deterministic — this is your single biggest credibility claim. A score reproduces exactly, every time.

SAY IT *“Deterministic means same product in, same score out — every time. No AI mood swings.”*

3 · Traceable vs. Black box

Traceable = you can see exactly *why* it produced an answer. A **black box** gives an answer with no explanation.

IN BARI Every score writes a “trace” — a record of which rules fired. It's deterministic *and* traceable; the two are different properties (reproducible vs. explainable) and we have both because it's rule-based code, not a model.

SAY IT *“Deterministic is about reproducibility; traceable is about explainability. We have both — because it's code, not a guess.”*

4 · Rule-based vs. Machine Learning (ML)

Rule-based = a human wrote explicit logic (“if sugar > X, then...”). **ML** = a model learned patterns from data and you can't fully see its reasoning.

ANALOGY *Rule-based is a recipe you can read line by line. ML is a chef who “just knows” — great food, but can't always explain the dish.*

IN BARI The scoring is rule-based on purpose. That choice is what buys you both determinism and traceability at once.

5 · Pipeline

A series of stages where each one's output is the next one's input — like an assembly line for data.

ANALOGY *A factory line: raw material → cut → assemble → paint → box. Each station does one job and passes it on.*

IN BARI Scrape → Extract → Understand → Score → Build → Website. Each stage hands a file to the next.

2 Data & how it's stored

↑ top

How information is shaped and saved so software can use it.

```

{  "name"   : "Cottage 5%"      ,      ← a "key" and its "value", in quotes = text
   "score"  : 72                ,      ← a number (no quotes)
   "grade"  : "B"               ,
   "sodium_mg" : null          ,      ← "null" = we don't have this value
   "tags"    : [ "high-protein" , "dairy" ]  ← a "list" = several values in [ ]
}
```

Figure B — The anatomy of a JSON file. It's just labelled boxes of data. This is exactly what one Bari product looks like.

JSON *"JAY-son"*

The universal text format for storing and passing structured data. It's just **labels and values** wrapped in brackets (see Figure B). Every system can read it.

ANALOGY *A filled-in form: each field has a label ("Name:") and a value ("Cottage 5%").*

IN BARI Every product, every score, every page is a JSON file. It's **the seam** — the one format where the backend hands data to the website.

SAY IT *"Each product is a JSON record — that's the single interface between our engine and our site."*

Key / Value · Field

A **key** is the label (`"score"`); its **value** is the data (`72`). One key+value is a **field**. It's how all structured data is organised.

NULL *"null"*

A deliberate marker meaning "there is no value here." Different from zero — zero is a number; null means *unknown / not captured*.

IN BARI Core to your integrity story: if a value wasn't scraped, the field is `null` and the page says "data could not be retrieved." You never replace null with a guess.

SAY IT *"Missing data stays null — we'd rather show a gap than invent a number."*

Schema

The agreed shape of your data — which fields exist and what type each is. A contract that keeps every record consistent.

ANALOGY *A blank form template: everyone fills in the same fields in the same order.*

Database vs. Files (and SQLite)

A **database** is specialised software for storing/querying lots of data (e.g. PostgreSQL, MySQL). **File-based** means storing data as plain files instead. **SQLite** is a tiny database that lives in a single file.

IN BARI Bari is mostly **file-based** (JSON) on purpose — so every result is reproducible and trackable in version control. A small SQLite file just records run lineage (which scrape produced which score).

SAY IT

“We chose a versioned file pipeline over a database — auditability matters more to us than query speed at this stage.”

3 The backend — making the data

↑ top

The tools that gather and judge the product data.

Python

A popular, readable programming language — the standard choice for data, science and automation work.

IN BARI The entire backend — scrapers, the Hebrew parser, the scoring engine — is Python. Chosen because the data and scientific tooling live there.

Web scraping

Automatically collecting data from websites by programmatically loading their pages and reading the content.

IN BARI How product data enters the system — scraping up to four Israeli retail chains directly. (No crowd databases, ever.)

Headless browser · Playwright

A **headless browser** is a real web browser running with no window, controlled by code. **Playwright** is the popular tool for driving one.

ANALOGY *A robot that opens the retailer's website exactly like a shopper would — but reads and saves the page automatically.*

IN BARI Our scrapers use Playwright so we see what a real customer sees, including pages that only load with a full browser.

OCR “optical character recognition” · Azure

OCR turns a photo of text into actual text a computer can read. **Azure** is Microsoft's cloud, where we run that OCR.

IN BARI For reading nutrition values off photographs of physical product labels when they aren't on the website.

Parsing · Enrichment

Parsing = reading messy raw text and pulling out the meaningful parts. **Enrichment** = adding derived insight on top (e.g. “this flour is whole-grain”).

IN BARI The “Understand” stage: a Hebrew ingredient parser detects flour type, additives, fermentation and protein source, then enriches the record before scoring.

Script · Batch job

A **script** is a small program that does one task. A **batch job** runs a task over many items at once.

IN BARI

Each category is scored by a batch run — one command processes the whole shelf and writes every product's result.

4 Testing & trust

↑ top

How engineers make sure a change doesn't silently break something — your strongest “we're serious” signals.

Unit test · pytest

A **unit test** is a small automated check that one piece of code does what it should. **pytest** is the standard tool for running them in Python.

IN BARI

The Hebrew label-reader alone has 64 automated checks. If a change breaks one, you know instantly.

Regression test · Baseline · Diff

A **regression** is when a new change accidentally breaks old behaviour. A **baseline** is a saved “known-good” result. A **diff** shows exactly what changed between two versions.

ANALOGY

Re-photographing every product after a change and laying it next to the old photo to spot anything that moved.

IN BARI

Change a rule → re-run all ~500 products → diff every score against the committed baseline. If something moved that shouldn't have, the gate fails. No eyeballing.

SAY IT

“Every rule change re-runs all 500 products and diffs against a baseline before it's allowed to count.”

Version control · Git · Commit

Git records every change to the code/data over time, so you can see history and undo anything. A **commit** is one saved snapshot.

ANALOGY *Track Changes for an entire project — every edit, who made it, and a button to roll back.*

IN BARI Because the data is files, every score is in version control — you can prove how any number came to be, historically.

Reproducibility

The property that anyone, anytime, can re-run the process and get the identical result. The gold standard for anything making a claim.

SAY IT

“Reproducibility is the whole game for a system that makes health-adjacent claims — and ours has it by design.”

5 The frontend — the website

↑ top

The technology that turns your JSON into the page a shopper sees.



Figure C — How a web page loads: the browser sends a **request**; the server sends back a **response** it then **renders**.

Next.js *"next J-S"*

A leading framework for building fast, modern websites. It's the structure the whole Bari site is built on.

ANALOGY *A pre-built chassis for a car — you don't forge the frame from scratch; you build on a proven one.*

IN BARI The site is Next.js. It reads the finished JSON and displays it — it never recalculates a score.

React · Component

React is the most popular tool for building interactive interfaces. A **component** is a reusable building block (a button, a card, a comparison row).

ANALOGY *LEGO bricks — build one "product card" once, reuse it for every product.*

IN BARI Every page is assembled from a small set of shared components — which is why all pages look consistent.

TypeScript

JavaScript (the language of the web) plus **type safety** — it catches whole classes of mistakes before the code ever runs.

ANALOGY *A form that rejects letters in a phone-number box — it stops bad data before it causes a problem.*

IN BARI The website is TypeScript, so the shape of every JSON field is checked — the site can't accidentally misread a score.

Tailwind (CSS)

CSS is what styles a web page (colours, spacing, fonts). **Tailwind** is a popular, consistent way of writing it.

IN BARI Keeps the visual design uniform across every page, via shared design tokens.

RTL *"right-to-left"*

Layout for languages that read right-to-left, like Hebrew and Arabic. Harder than it sounds — everything mirrors.

IN BARI The whole site is built RTL-correct for Hebrew — a real piece of engineering and part of your local moat.

Rendering · DOM · Responsive

Rendering = turning code/data into the visible page. The **DOM** is the page's live structure in the browser. **Responsive** = the layout adapts to any screen size.

IN BARI Pages are mobile-first and responsive — the target is a shopper understanding the shelf in 15–20 seconds on a phone.

Server-side / Static rendering (SSR / SSG)

Two ways to build a page: **SSR** builds it fresh on the server per visit; **SSG** builds it once ahead of time so it loads instantly. Most sites mix both.

IN BARI Comparison pages can be pre-built from the JSON, so they're fast and cheap to serve.

Deploy · Vercel

Deploying = publishing the website so the public can reach it. **Vercel** is the hosting service built for Next.js sites.

IN BARI “Re-publishing” a category = regenerate its JSON, then redeploy the site. No hand-editing of live pages.

Accessibility · WCAG · Contrast

Making a site usable by everyone, including low-vision users. **WCAG** is the standard; **contrast** is whether text is readable against its background.

IN BARI A design reviewer checks every page for contrast and accessibility before it ships.

Where AI sits in Bari — and, just as importantly, where it does not.

The line that wins the room: AI in Bari does two narrow jobs — helping *read* messy Hebrew labels, and helping *draft* consumer copy. It **never** sets a nutrition value or a score. The verdict is deterministic code. That's what separates you from an “AI nutrition app.”

LLM “large language model” • Model

An AI trained on huge amounts of text that can read and write language (ChatGPT, Claude, etc.). A **model** is the trained system itself.

IN BARI Used for language tasks (parsing help, copywriting) — outside the scoring path, always checked downstream.

Prompt

The instruction you give an AI model to get an output.

ANALOGY *The brief you hand a freelancer — the clearer the brief, the better the result.*

Hallucination

When an AI confidently states something false. The core reason you can't let an LLM produce nutrition facts or scores.

SAY IT

“We never let a model invent a number — that's exactly where AI hallucinates. Numbers come from the scrape and deterministic code.”

Agent · Orchestration

An **agent** is an AI given a job and some tools to do it. **Orchestration** is coordinating several agents/roles toward one outcome.

IN BARI Production runs as a coordinated set of specialist roles (data, research, content, QA) with one lead checking each result against evidence before it counts.

Training vs. Inference

Training = teaching a model from data (slow, expensive, one-time). **Inference** = using the finished model to get an answer (fast).

IN BARI Useful to know if the engineer asks. Today Bari doesn't train its own model; the future “personalisation engine” is where ML/training could enter — and even then, it sits beside the deterministic score, not inside it.

7 Engineering glue & practices

↑ top

The connective concepts that come up in any technical conversation.

API *“application programming interface”*

A defined way for two systems to talk to each other — one program asks, another answers, in an agreed format.

ANALOGY *A restaurant menu: a fixed list of things you can order and what you'll get back. You don't go into the kitchen.*

IN BARI Relevant to the plan: selling the scored data to a health fund could mean exposing it through an API their app calls.

Repository (“repo”) · Monorepo

A **repo** is the project's folder of code, tracked by Git. A **monorepo** keeps several related projects in one repo.

IN BARI Bari is a monorepo: the data/scoring workspace and the website live together but stay cleanly separated.

Feature flag · Rollback

A **feature flag** is an on/off switch for a change, so you can release it safely or turn it off instantly. A **rollback** undoes a change.

IN BARI New scoring rules ship behind a flag (default off) with a defined rollback — so nothing changes published scores by accident.

SAY IT *“Every scoring change is behind a flag with a rollback — we never let an experiment touch live scores silently.”*

Gate · Conformance check

A **gate** is an automated checkpoint that blocks a change unless it passes. A **conformance check** verifies something still follows the required structure.

IN BARI Before a category goes live it must pass gates (data integrity, no regressions, the red-team review). A conformance check confirms every category will still re-flow when the engine changes.

Edge case

An unusual input that breaks normal assumptions — the thing that exposes weak software.

IN BARI e.g. a product with a missing panel, or a whole-food fat like almonds that's calorie-dense but healthy. The engine has deliberate handling (floors) for these.

Internal terms — mainly for *your* understanding. Most of these stay in-house; don't lead with them to outsiders.

Trace

The full record a score writes — every rule that fired, every cap, every dimension's contribution. The source of “traceability.”

Dimension

One of the ten angles the engine scores a product on (processing, nutrient density, etc.), each 0–100.

Cap · Floor

A **cap** is a hard ceiling on the score (serious problems can't be outweighed). A **floor** is a protective minimum (whole foods aren't punished for natural traits). These do the heavy lifting at the extremes — more than the weights.

Score · Grade

The **score** is the 0–100 number; the **grade** (S–E) is the band it falls into. The grade is for the shopper; the score drives the ranking.

BSIP *internal — keep in-house*

Your internal name for the pipeline's three stages (extraction → enrichment → scoring). Useful shorthand internally; to outsiders just say “our pipeline.”

TERM	IN ONE LINE
Backend	The hidden half — data & scoring (Python).
Frontend	The visible half — the website (Next.js).
Deterministic	Same input → same output, always.
Traceable	You can see exactly why it gave that answer.
Rule-based	Explicit human-written logic, not a learned model.
Pipeline	An assembly line of data stages.
JSON	The universal “labelled boxes” text format for data.
Null	“No value” — we never guess to fill it.
Schema	The agreed shape every record follows.
Python	The language of the backend.
Scraping	Collecting data from websites automatically.
Playwright	A robot browser that loads pages like a shopper.
OCR	Turning a photo of text into readable text.
Unit test / pytest	Automated checks that code works.
Regression test	Re-run everything; flag anything that moved.
Baseline / diff	Known-good result; the change between versions.
Git	Track-changes for the whole project.
Reproducibility	Anyone can re-run it and get the same result.
Next.js	The framework the website is built on.
React / Component	Interface tool / a reusable building block.
TypeScript	Web language with mistake-catching built in.
Tailwind	Consistent way to style the pages.

RTL	Right-to-left layout (Hebrew).
Render / DOM	Turning data into the visible page / its live structure.
SSR / SSG	Build a page per-visit / build it ahead of time.
Deploy / Vercel	Publish the site / the host it runs on.
LLM / Model	An AI that reads & writes language.
Hallucination	When AI confidently says something false.
Agent / Orchestration	An AI with a job / coordinating several of them.
API	A defined way for two systems to talk.
Monorepo	Several projects in one tracked folder.
Feature flag / Rollback	On-off switch for a change / undo it.
Gate / Conformance	Automated checkpoint that blocks bad changes.

10 Sound fluent · avoid these slips

[↑ top](#)

Three sentences that signal you know your architecture:

"It's deterministic — same input, same score, with a full trace."

"The engine and the site meet only through generated JSON."

"Every rule change is regression-gated against all 500 products."

Don't say / watch for:

- **Don't** call the scoring "AI" or "the algorithm learns" — it's rule-based and doesn't learn. Saying otherwise undercuts your whole credibility story.
- **Don't** collapse "deterministic" and "traceable" into one word — they're two different claims (reproducible vs. explainable). You have both; say both.

- **Don't** overclaim productionization. The honest line: “the engine and pipeline run end-to-end and are proven; hardening it into a clean service is exactly what a technical partner would own.”
- **Don't** guess a number you don't know. “I'd have to check the exact figure” beats a confident wrong answer — every engineer respects that more.